TURNING
QUESTIONS
INTO QUERIES

HomeCoded

Joshua Gonzalez

Index

- Mission Statement
- About Us
- Business Rules
- ERD
- Simple Queries
- Complex Queries
- Customer Invoice
- Calculation
- Sub Queries

TechNest

- TechNest's mission is to bridge the gap between the past and the future, offering furniture that embodies the comfort of tradition while embracing the possibilities of technology.

Link For SQL Commands

About Us

- At TechNest, our journey began with a simple yet profound realization: the way we furnish our homes can greatly influence our quality of life. Founded by twin siblings Luke and Leia Voss in Scranton, Pennsylvania, we have been dedicated since 1997 to bringing comfort and innovation together. Our meticulously crafted furniture pieces, ranging from our self-cleaning tables to bedframes with integrated projectors, aim to create environments that inspire relaxation, creativity, and connection. As a proud local business, we take our responsibility to the community seriously, striving to serve our neighbors with furniture that is both stylish and functional. We believe that comfort should never be compromised by style, and that innovation should be accessible in every room. Our friendly team is here to help, offering personalized service and high quality maintenance to ensure you receive the best value. Whether you're looking to lavish your home or upgrade your workplace, we are excited to be your trusted partner in sophistication, committed to enhancing lives through thoughtful design and community involvement.

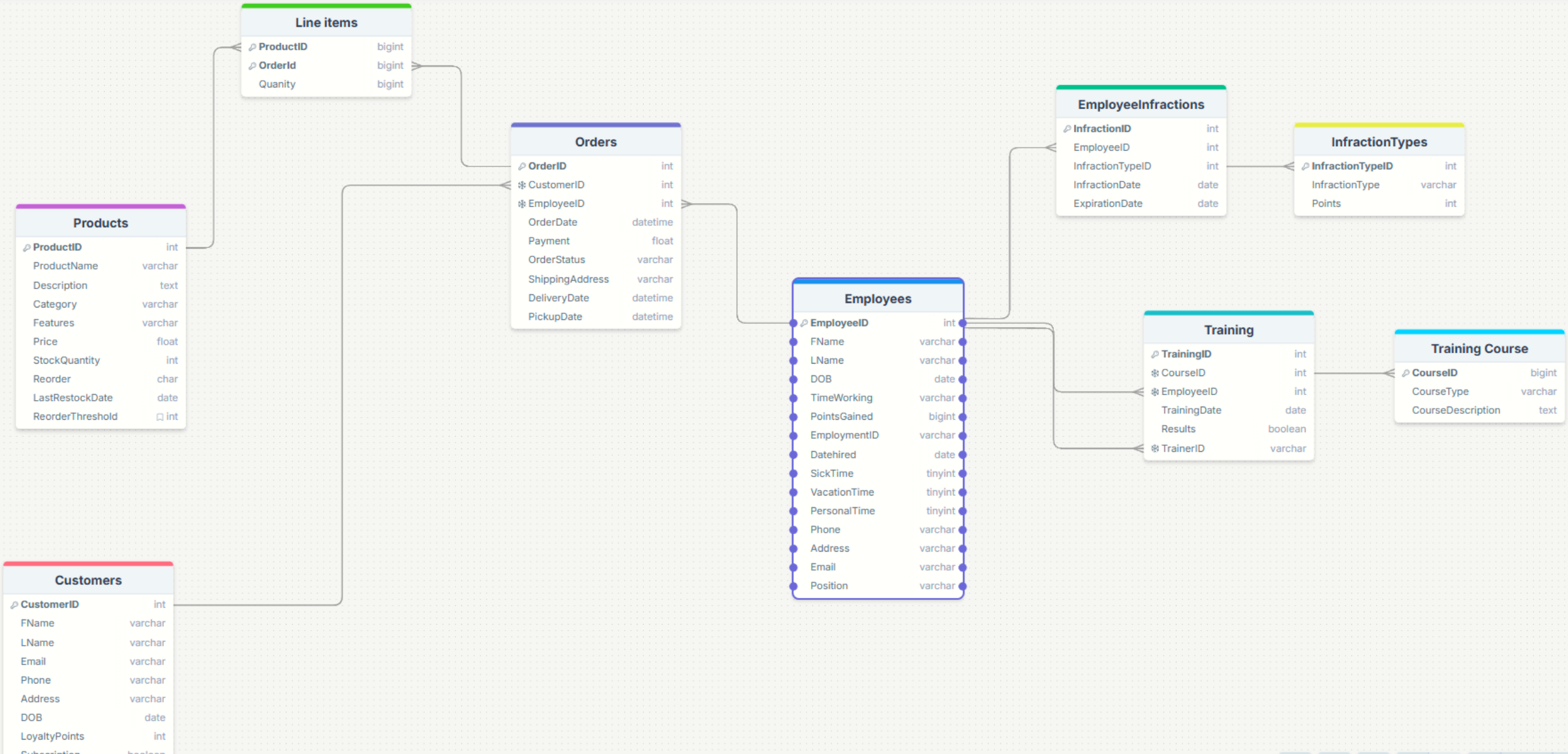
Rules

1. Business Hours - Mon–Fri: 9 AM to 8 PM, Sun: 9 AM to 5 PM
2. Confidentiality & Data Protection - Customer and company information must be handled securely and not shared outside authorized personnel.
3. Copyright & Intellectual Property - All product designs, images, and promotional materials, including but not limited to logos, photographs, and descriptions, are the exclusive property of the store and are protected by copyright law. Unauthorized reproduction, distribution, or use of these materials without express written consent is prohibited.
4. Pricing & Promotions - Discounts and promotions apply only during advertising and cannot be combined with other offers unless stated otherwise (Black Friday, Labor Day, Lazy Day - August 10).
5. **Staff Training - Staff must attend seminars and training sessions to remain up-to-date on policies and product knowledge. The training must be passed and approved by a trainer.**
6. **Uniform Policy - Business casual attire is required for all employees.**
7. Breaks and Lunches - Staff are entitled to two 15-minute breaks and one 45-minute lunch per eight-and-a-half-hour shift.
8. **Attendance Policy -Tardiness (1-2 points) and no-call/no-shows (6 points) are tracked using a point system: verbal warning at 6 points, written at 9, suspension at 12, and termination at 15. Points expire after a year.**
9. Promotion Eligibility - Employees must work a minimum of one year before becoming eligible for a higher position.
10. **Vacation Time Eligibility - Employees must work at least 6 months before being eligible for paid time off.**

Rules

11. **Total Time Off Limitations - Total time off cannot exceed 30 days, sick days are limited to 9, vacation time is limited to 14, and personal time is limited to 7.**
12. **Commission Bonus - The highest-earning commission employee receives a $1,000 bonus.**
13. Single Job Title - Employees can only belong to one department and cannot have multiple job titles.
14. Minors' Employment - Minors (under 18) are permitted to work part-time hours only, per labor regulations.
15. Employee Orders - Employees may not process their orders if they buy something under their name to prevent misuse of privileges.
16. Unique Emails/Phone Numbers - Both employee and customer email and phone numbers must be unique and cannot match existing accounts.
17. **Loyalty Point Limitation - Customers' loyalty points are capped at 3,000. A reminder to use points will be sent once a customer reaches the 2,000-point threshold.**
18. Shipping Policy – All orders are shipped via USPS. Shipping costs are calculated based off USPS rates and are outside TechNest's control.
19. **Return Policy - Returns of new and unopened products are allowed within 365 days for a full refund. Open products can be returned within 180 days.**
20. **Restock – Products are expected to be at their restock threshold at the end of every month. Weekly checks will be done to ensure products are up to code.**
21. Payment Limit - Each customer order will be limited to a single payment method; multiple cards will not be accepted for an order.
22. **Membership - Each customer must be a member at TechNest to make a purchase. If you are not a member, you can sign up for one at checkout with your order.**

ERD Diagram


Line items
ProductID bigint
OrderId bigint
Quanity bigint
Orders
OrderID int
CustomerID int
EmployeeID int
OrderDate datetime
Payment float
OrderStatus varchar
ShippingAddress varchar
DeliveryDate datetime
PickupDate datetime
EmployeeInfractions
InfractionID int
EmployeeID int
InfractionTypeID int
InfractionDate date
ExpirationDate date
InfractionTypes
InfractionTypeID int
InfractionType varchar
Points int
Products
ProductID int
ProductName varchar
Description text
Category varchar
Features varchar
Price float
StockQuantity int
Reorder char
LastRestockDate date
ReorderThreshold int
Employees
EmployeeID int
FName varchar
LName varchar
DOB date
TimeWorking varchar
PointsGained bigint
EmploymentID varchar
Datehired date
SickTime tinyint
VacationTime tinyint
PersonalTime tinyint
Phone varchar
Address varchar
Email varchar
Position varchar
Training
TrainingID int
CourseID int
EmployeeID int
TrainingDate date
Results boolean
TrainerID varchar
Training Course
CourseID bigint
CourseType varchar
CourseDescription text
Customers
CustomerID int
FName varchar
LName varchar
Email varchar
Phone varchar
Address varchar
DOB date
LoyaltyPoints int

Created Tables

```
CREATE TABLE Orders (
OrderID INT NOT NULL
AUTO_INCREMENT PRIMARY
KEY,
CustomerID INT,
EmployeeID INT,
OrderDate DATETIME,
Payment VARCHAR(50),
OrderStatus VARCHAR(50),
ShippingAddress
VARCHAR(50), DeliveryDate
DATETIME,
PickupDate DATETIME,
FOREIGN KEY (CustomerID)
REFERENCES
Customers(CustomerID),
FOREIGN KEY (EmployeeID)
REFERENCES
Employees(EmployeeID) );
```

```
CREATE TABLE Employees (
   EmployeeID INT NOT NULL
AUTO_INCREMENT PRIMARY KEY,
   FName VARCHAR(255),
   LName VARCHAR(255),
   DOB DATE,
   TimeWorking VARCHAR(255),
   PointsGained INT,
   Employment VARCHAR(255),
   DateHired DATE,
   SickTime TINYINT,
   VacationTime TINYINT,
   PersonalTime TINYINT,
   Phone VARCHAR(255),
   Address VARCHAR(255),
   Email VARCHAR(255),
   Position VARCHAR(255)
);
```

```
CREATE TABLE Customers (
CustomerID INT NOT NULL
AUTO_INCREMENT PRIMARY KEY,
FName VARCHAR(255),
LName VARCHAR(255),
Email VARCHAR(255),
Phone VARCHAR(255),
Address VARCHAR(255),
DOB DATE,
LoyaltyPoints INT,
Subscription TINYINT(1),
Registration DATE );
```

Insert Statements

- INSERT INTO Employees (FName, LName, DOB, TimeWorking, PointGained, Employment, DateHired, SickTime, VacationTime, PersonalTime, Phone, Address, Email, Position) VALUES ('John', 'Doe', '1985-06-15', '3 years', 8.5, 'Full-Time', '2022-05-10', 15.0, 3, 10, 5, '(555) 773-5819', '123 Elm St', 'jdoe5268@gmail.com', 'Senior Customer Service Representative');
  - UPDATE Employees SET TimeWorking = CONCAT(TIMESTAMPDIFF(YEAR, DateHired, CURDATE()), ' years, ', TIMESTAMPDIFF(MONTH, DateHired, CURDATE()) % 12, ' months');
- INSERT INTO Customers (FName, LName, Email, Phone, Address, DOB, LoyaltyPoints, Subscription, Registration) VALUES ('Grace', 'Hawkins', 'barbermovie@gmail.com', '890-1053', '66 Wolfpack Way', '1991-01-22', 500, 1, '2022-03-03');
- INSERT INTO Orders (CustomerID, EmployeeID, OrderDate, Payment, OrderStatus, ShippingAddress, DeliveryDate, PickupDate) VALUES (1, 4, '2023-05-14 10:23:00', 'Credit Card', 'Delivered', '12 Riverbend Dr', '2023-05-20 14:00:00');
- INSERT INTO LineItems (ProductID, OrderID, Quantity) VALUES (1, 150, 2);

Simple Queries

1. Loyalty Points Check

- This query will show all customers that are approaching the max loyalty points.
- The company will use this information to contact the customers and notify them to use their points. As per **rule 17** customers are notified when they are at 2,000 points.

```
SELECT fName AS ‘First Name’, lName AS 'Last Name', LoyaltyPoints,
email as ‘Contact Info’
FROM Customers
WHERE LoyaltyPoints >= 2000;
```

SQL Statement: SELECT fName AS 'First Name', lName AS 'Last Name', LoyaltyPoints, email as 'Contact Info' FROM Customers WHERE LoyaltyPoints >= 2000;

Output:

First Name	Last Name	LoyaltyPoints	Contact Info
Lucas	Miller	2000	lucas.miller@gmail.com
Tom	Holland	2000	tom.holland@totallyreal.com
Evelyn	Reyes	2000	evelyn.reyes@hotmail.com
Scarlett	Johansson	2000	scarlett.johansson@gmail.com
Jennifer	Lawrence	2000	jennifer.lawrence@gmail.com
Isabella	Martinez	2000	isabella.martinez@gmail.com
Henry	Baker	2000	henry.baker@gmail.com
Ella	Young	2000	ella.young@outlook.com
Sebastian	Wright	2000	seb.wright@gmail.com

Simple Queries – Date Range

2. Return Eligibility

- This query will show all orders that are eligible for return.
- The company will use this information to accept returns from orders from the accepted range. As per **rule 19**, customers are allowed 180 days for unopened/opened boxes and 365 for unopened boxes.

```
SELECT OrderID, OrderDate,
  CASE
    WHEN OrderDate >= CURRENT_DATE - INTERVAL '180' DAY THEN 'Open/Unopened'
    WHEN OrderDate >= CURRENT_DATE - INTERVAL '365' DAY THEN
'Unopened'
    ELSE 'Older than 365 Days'
  END AS 'Return Eligibility'
FROM Orders
WHERE OrderDate >= CURRENT_DATE - INTERVAL '365' DAY;
```

Simple Queries – Date Range

2. Return Eligibility

SQL Command:

```
  END AS 'Return Eligibility'

FROM Orders

WHERE OrderDate >= CURRENT_DATE - INTERVAL '365' DAY;
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT OrderID, OrderDate, CASE WHEN OrderDate >= CURRENT_DATE - INTERVAL '180' DAY THEN 'Open/Unopened' WHEN OrderDate >= CURRENT_DATE - INTERVAL '365' DAY THEN 'Unopened' ELSE 'Older than 365 Days' END AS 'Return Eligibility' FROM Orders WHERE OrderDate >= CURRENT_DATE - INTERVAL '365' DAY;

Output:

OrderID	OrderDate	Return Eligibility
158	2024-07-11 14:15:00	Unopened
161	2025-04-01 14:00:00	Open/Unopened
162	2025-04-07 11:30:00	Open/Unopened
163	2025-04-10 13:45:00	Open/Unopened
164	2025-04-28 09:30:00	Open/Unopened
168	2025-03-18 10:00:00	Open/Unopened
169	2024-06-30 13:00:00	Unopened
173	2025-04-12 12:15:00	Open/Unopened
174	2025-04-30 15:00:00	Open/Unopened
175	2024-09-22 08:45:00	Unopened
190	2025-05-08 23:16:54	Open/Unopened
191	2025-05-08 23:18:26	Open/Unopened
192	2025-05-08 23:18:58	Open/Unopened

Simple Queries – SQL Function

3. Time Off Eligibility

- This query will check that all employees with any time off have worked at least 6 months.
- The company will use this information to ensure time off is going to the right employees as per **rule 10** that requires 6 months of working before PTO.

```
SELECT EmployeeID, FName 'First Name', LName 'Last Name', SickTime, VacationTime,
PersonalTime, DateHired, DATEDIFF(CURDATE(), DateHired) AS DaysWorked,
    CASE
        WHEN (SickTime > 0 OR VacationTime > 0 OR PersonalTime > 0)
            AND DATEDIFF(CURDATE(), DateHired) < 180
        THEN 'Violation'
        ELSE 'Valid'
    END AS Status
FROM Employees;
```

Simple Queries – SQL Function

SQL Command:

```
SELECT EmployeeID, FName 'First Name', LName 'Last Name', SickTime, VacationTime,
PersonalTime, DateHired, DATEDIFF(CURDATE(), DateHired) AS DaysWorked,
    CASE
        WHEN (SickTime > 0 OR VacationTime > 0 OR PersonalTime > 0)
            AND DATEDIFF(CURDATE(), DateHired) < 180
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT EmployeeID, FName 'First Name', LName 'Last Name', SickTime, VacationTime, PersonalTime, DateHired, DATEDIFF(CURDATE(), DateHired) AS DaysWorked, CASE WHEN (SickTime > 0 OR VacationTime > 0 OR PersonalTime > 0) AND DATEDIFF(CURDATE(), DateHired) < 180 THEN 'Violation' ELSE 'Valid' END AS Status FROM Employees;

Output:

EmployeeID	First Name	Last Name	SickTime	VacationTime	PersonalTime	DateHired	DaysWorked	Status
1	John	Doe	7	13	5	2022-05-10	1095	Valid
2	Jane	Smith	6	10	5	2021-11-15	1271	Valid
3	Alice	Johnson	8	10	7	2023-02-20	809	Valid
4	Bob	Williams	9	2	5	2019-04-18	2213	Valid
5	Charlie	Brown	6	6	5	2015-08-25	3545	Valid
6	David	Miller	2	8	6	2024-01-10	485	Valid
7	Eva	Davis	8	4	1	2020-06-14	1790	Valid
8	Frank	Garcia	3	10	3	2012-03-05	4813	Valid
9	Grace	Martinez	7	9	7	2024-02-01	463	Valid
10	Hannah	Lopez	7	10	7	2021-09-17	1330	Valid
11	Isabella	Reed	5	1	7	2022-04-12	1123	Valid
12	Liam	Nguyen	9	11	6	2023-06-20	689	Valid
13	Sophia	Allen	8	10	7	2019-08-01	2108	Valid
14	Noah	Lee	9	10	4	2021-05-10	1460	Valid
15	Chloe	White	7	2	6	2024-03-01	434	Valid
16	Ethan	Baker	4	11	6	2016-07-18	3217	Valid
17	Avery	Hernandez	5	1	7	2020-09-10	1702	Valid
18	Jackson	Wright	8	14	3	2022-06-22	1052	Valid
19	Lily	Young	1	12	7	2018-01-14	2672	Valid
20	Mason	Scott	8	10	6	2021-11-25	1261	Valid
21	Ben	Parker	0	0	0	2025-02-01	97	Valid
22	Luna	Martell	0	0	0	2025-03-10	60	Valid
23	Marcus	Thompson	6	8	7	1999-03-15	9552	Valid
24	Diane	Brooks	9	13	7	2002-06-10	8369	Valid
25	Carlos	Rivera	8	3	5	2005-02-28	7375	Valid
26	Sam	Taylor	6	6	2	1998-04-01	9900	Valid

Simple Queries – GroupBy

4. Membership Check

- This query will count all customers with a membership.
- The company will use this information to cross examine if the number of customers matches the number of memberships. As per **rule 22** every customer needs a membership.

```
SELECT Subscription, COUNT(*) AS 'Customers with Membership'
FROM Customers
GROUP BY Subscription;
```

SQL Statement: SELECT Subscription, COUNT(*) AS 'Customers with Membership' FROM Customers GROUP BY Subscription;

Output:

Subscription	Customers with Membership
1	30

Simple Queries

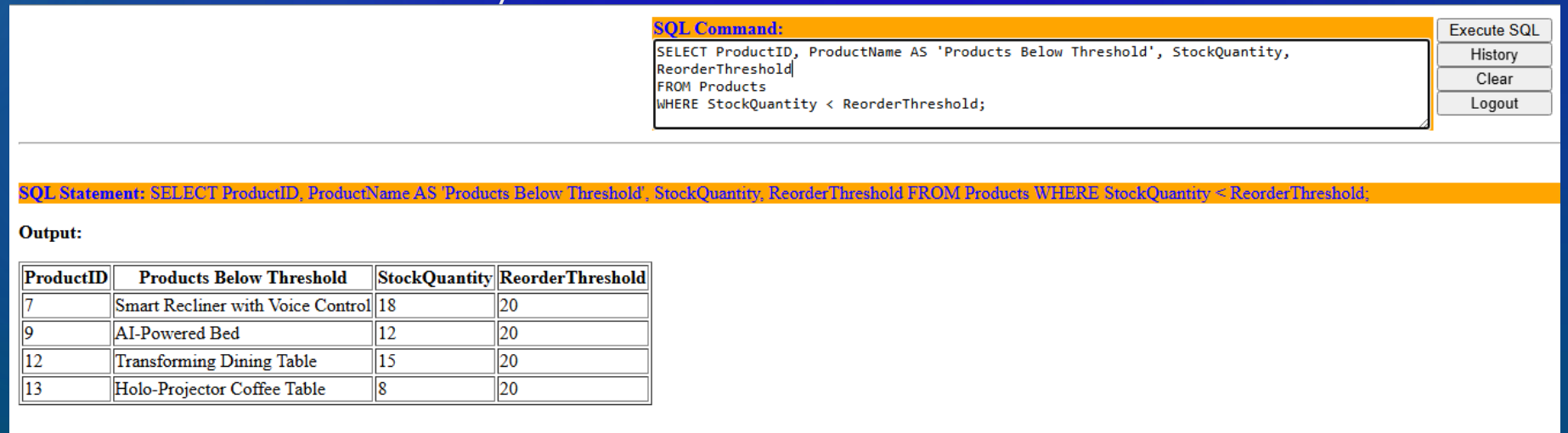
5. Restock Threshold

- This query will list any product that is currently not meeting the restock threshold.
- The company will use this information to order more products.

```
SELECT ProductID, ProductName AS 'Products Below Threshold',
StockQuantity, ReorderThreshold

FROM Products

WHERE StockQuantity < ReorderThreshold;
```



SQL Command:

```
SELECT ProductID, ProductName AS 'Products Below Threshold', StockQuantity,
ReorderThreshold
FROM Products
WHERE StockQuantity < ReorderThreshold;
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT ProductID, ProductName AS 'Products Below Threshold', StockQuantity, ReorderThreshold FROM Products WHERE StockQuantity < ReorderThreshold;

Output:

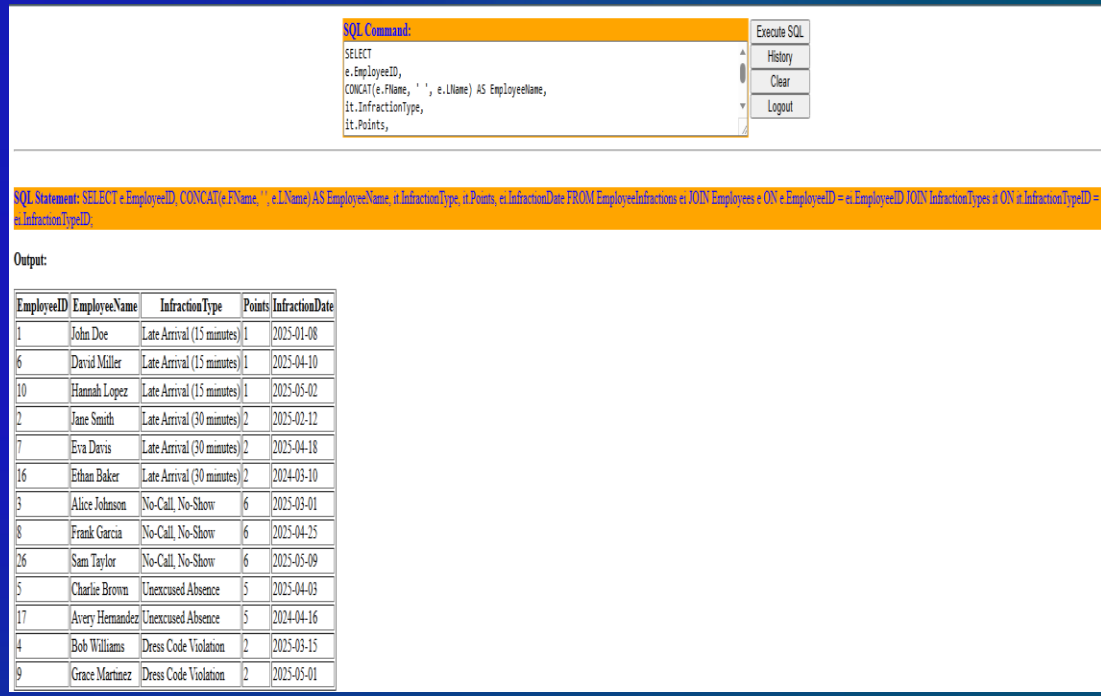
ProductID	Products Below Threshold	StockQuantity	ReorderThreshold
7	Smart Recliner with Voice Control	18	20
9	AI-Powered Bed	12	20
12	Transforming Dining Table	15	20
13	Holo-Projector Coffee Table	8	20

Complex Queries 1

Employee Infraction

- This query will list employees with penalty points, how many they have, and when they got them.
- The company will use this information to track which employees are getting close to the 15-point termination.

```
SELECT
e.EmployeeID,
CONCAT(e.FName, ' ', e.LName) AS EmployeeName,
it.InfractionType,
it.Points,
ei.InfractionDate
FROM EmployeeInfractions ei
JOIN Employees e ON e.EmployeeID = ei.EmployeeID
JOIN InfractionTypes it ON it.InfractionTypeID = ei.InfractionTypeID;
```



SQL Command:

```
SELECT
e.EmployeeID,
CONCAT(e.FName, ' ', e.LName) AS EmployeeName,
it.InfractionType,
it.Points,
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT e.EmployeeID, CONCAT(e.FName, ' ', e.LName) AS EmployeeName, it.InfractionType, it.Points, ei.InfractionDate FROM EmployeeInfractions ei JOIN Employees e ON e.EmployeeID = ei.EmployeeID JOIN InfractionTypes it ON it.InfractionTypeID = ei.InfractionTypeID;

Output:

EmployeeID	EmployeeName	InfractionType	Points	InfractionDate
1	John Doe	Late Arrival (15 minutes)	1	2025-01-08
6	David Miller	Late Arrival (15 minutes)	1	2025-04-10
10	Hannah Lopez	Late Arrival (15 minutes)	1	2025-05-02
2	Jane Smith	Late Arrival (30 minutes)	2	2025-02-12
7	Eva Davis	Late Arrival (30 minutes)	2	2025-04-18
16	Ethan Baker	Late Arrival (30 minutes)	2	2024-03-10
3	Alice Johnson	No-Call, No-Show	6	2025-03-01
8	Frank Garcia	No-Call, No-Show	6	2025-04-25
26	Sam Taylor	No-Call, No-Show	6	2025-05-09
5	Charlie Brown	Unexcused Absence	5	2025-04-03
17	Avery Hernandez	Unexcused Absence	5	2024-04-16
4	Bob Williams	Dress Code Violation	2	2025-03-15
9	Grace Martinez	Dress Code Violation	2	2025-05-01

Complex Queries 2

Training Course Info

- This query will list employees with their training, trainer, and the results.
- The company will use this information to track which employees need training.

```
SELECT
    CONCAT(e.FName, ' ', e.LName) AS EmployeeName,
    tc.CourseDesc AS TrainingCourse,
    tc.CourseType AS CourseType,
    CONCAT(t.FName, ' ', t.LName) AS TrainerName,
    CASE
        WHEN tr.Results = 1 THEN 'Pass'
        WHEN tr.Results = 0 THEN 'Fail'
        ELSE 'Unknown'
    END AS TrainingResult
FROM
    Training tr
JOIN
    Employees e ON tr.EmployeeID =
e.EmployeeID
JOIN
    TrainingCourse tc ON tr.CourseID =
tc.CourseID
JOIN
    Employees t ON tr.TrainerID = t.EmployeeID
ORDER BY
    tr.TrainingID;
```

Complex Queries 2

SQL Command:

```
    TrainingCourse tc ON tr.CourseID = tc.CourseID
JOIN
    Employees t ON tr.TrainerID = t.EmployeeID
ORDER BY
    tr.TrainingID;
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT CONCAT(e.FName, ' ', e.LName) AS EmployeeName, tc.CourseDesc AS TrainingCourse, tc.CourseType AS CourseType, CONCAT(t.FName, ' ', t.LName) AS TrainerName, CASE WHEN tr.Results = 1 THEN 'Pass' WHEN tr.Results = 0 THEN 'Fail' ELSE 'Unknown' END AS TrainingResult FROM Training tr JOIN Employees e ON tr.EmployeeID = e.EmployeeID JOIN TrainingCourse tc ON tr.CourseID = tc.CourseID JOIN Employees t ON tr.TrainerID = t.EmployeeID ORDER BY tr.TrainingID;

Output:

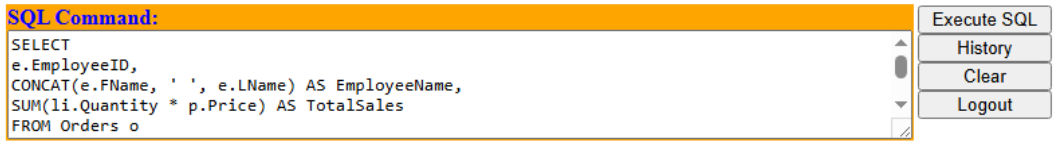
EmployeeName	TrainingCourse	CourseType	TrainerName	TrainingResult
John Doe	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Jane Smith	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Alice Johnson	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Bob Williams	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Charlie Brown	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
David Miller	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Eva Davis	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Frank Garcia	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Grace Martinez	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Hannah Lopez	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Isabella Reed	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Liam Nguyen	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Sophia Allen	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Noah Lee	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Chloe White	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Ethan Baker	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Avery Hernandez	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Jackson Wright	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Lily Young	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Mason Scott	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Ben Parker	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Luna Martell	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Diane Brooks	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
Carlos Rivera	An introductory course designed for new employees, providing them with an overview of TechNest's mission, values, and products.	Basic Enrollment Training	Sam Taylor	Pass
John Doe	Training focused on providing excellent customer service for both online and in-store customers.	Customer Service Excellence	Sam Taylor	Pass
John Doe	An in-depth course on the features, benefits, and specifications of TechNest's tech-based furniture products.	Product Knowledge	Sam Taylor	Pass
Jane Smith	Training focused on providing excellent customer service for both online and in-store customers.	Customer Service Excellence	Sam Taylor	Pass

Complex Queries 3

Best Salesperson (Current Year)

- This query will reveal the highest sales commission by an employee.
- The company will use this information to find out who the best seller is for their raise as per **rule 12** that gives a bonus for our highest commission.

```
SELECT
e.EmployeeID,
CONCAT(e.FName, ' ', e.LName) AS EmployeeName,
SUM(li.Quantity * p.Price) AS TotalSales
FROM Orders o
JOIN Employees e ON o.EmployeeID = e.EmployeeID
JOIN LineItems li ON o.OrderID = li.OrderID
JOIN Products p ON li.ProductID = p.ProductID
WHERE YEAR(o.OrderDate) = YEAR(CURDATE())
GROUP BY e.EmployeeID, EmployeeName
ORDER BY TotalSales DESC
LIMIT 1;
```



SQL Statement: SELECT e.EmployeeID, CONCAT(e.FName, ' ', e.LName) AS EmployeeName, SUM(li.Quantity * p.Price) AS TotalSales FROM Orders o JOIN Employees e ON o.EmployeeID = e.EmployeeID JOIN LineItems li ON o.OrderID = li.OrderID JOIN Products p ON li.ProductID = p.ProductID WHERE YEAR(o.OrderDate) = YEAR(CURDATE()) GROUP BY e.EmployeeID, EmployeeName ORDER BY TotalSales DESC LIMIT 1;

Output:

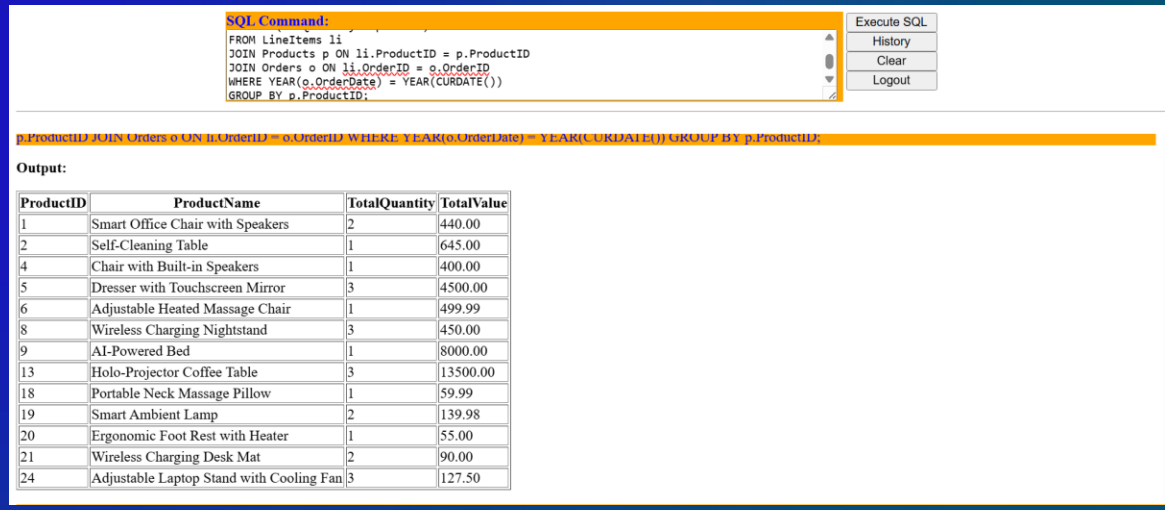
EmployeeID	EmployeeName	TotalSales
4	Bob Williams	21999.99

Complex Queries 4

Total Value

- This query will show total quantity and total value of products sold per product this year.
- The company will use this information to find what items are selling the most or if another product needs to pushed.

```
SELECT
    p.ProductID,
    p.ProductName,
    SUM(li.Quantity) AS TotalQuantity,
    SUM(li.Quantity * p.Price) AS TotalValue
FROM LineItems li
JOIN Products p ON li.ProductID = p.ProductID
JOIN Orders o ON li.OrderID = o.OrderID
WHERE YEAR(o.OrderDate) = YEAR(CURDATE())
GROUP BY p.ProductID;
```



SQL Command:

```
FROM LineItems li
JOIN Products p ON li.ProductID = p.ProductID
JOIN Orders o ON li.OrderID = o.OrderID
WHERE YEAR(o.OrderDate) = YEAR(CURDATE())
GROUP BY p.ProductID;
```

Execute SQL | History | Clear | Logout

p.ProductID JOIN Orders o ON li.OrderID = o.OrderID WHERE YEAR(o.OrderDate) = YEAR(CURDATE()) GROUP BY p.ProductID;

Output:

ProductID	ProductName	TotalQuantity	TotalValue
1	Smart Office Chair with Speakers	2	440.00
2	Self-Cleaning Table	1	645.00
4	Chair with Built-in Speakers	1	400.00
5	Dresser with Touchscreen Mirror	3	4500.00
6	Adjustable Heated Massage Chair	1	499.99
8	Wireless Charging Nightstand	3	450.00
9	AI-Powered Bed	1	8000.00
13	Holo-Projector Coffee Table	3	13500.00
18	Portable Neck Massage Pillow	1	59.99
19	Smart Ambient Lamp	2	139.98
20	Ergonomic Foot Rest with Heater	1	55.00
21	Wireless Charging Desk Mat	2	90.00
24	Adjustable Laptop Stand with Cooling Fan	3	127.50

Complex Queries 5

Orders Per Employee

- This query will display all the employees with their sales amount.
- The company will use this to track the performance of salespeople.

```
SELECT e.EmployeeID, CONCAT(e.FName, ' ',
e.LName) AS EmployeeName,
 COUNT(o.OrderID) AS TotalOrders
FROM Employees e
JOIN Orders o ON e.EmployeeID = o.EmployeeID
GROUP BY e.EmployeeID, e.FName, e.LName
ORDER BY TotalOrders DESC;
```

SQL Command:

```
SELECT
  e.EmployeeID,
  CONCAT(e.FName, ' ', e.LName) AS EmployeeName,
  COUNT(o.OrderID) AS TotalOrders
FROM Employees e
JOIN Orders o ON e.EmployeeID = o.EmployeeID
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT e.EmployeeID, CONCAT(e.FName, ' ', e.LName) AS EmployeeName, COUNT(o.OrderID) AS TotalOrders FROM Employees e JOIN Orders o ON e.EmployeeID = o.EmployeeID GROUP BY e.EmployeeID, e.FName, e.LName ORDER BY TotalOrders DESC;

Output:

EmployeeID	EmployeeName	TotalOrders
4	Bob Williams	11
6	David Miller	9
7	Eva Davis	9
13	Sophia Allen	9
11	Isabella Reed	3
21	Ben Parker	1
22	Luna Martell	1

Customer Invoice

Order Info

- This query retrieves the full names of the customer and employee, product name, unit price, quantity, calculated total, applied discount, and the final discounted total.
- The company will use this information for ordering.

```
SELECT o.OrderID, o.OrderDate,
o.Payment, o.OrderStatus,
o.ShippingAddress, o.PickupDate,
o.DeliveryDate, CONCAT(c.FName, ' ',
c.LName) AS 'Customer Name',
CONCAT(e.FName, ' ', e.LName) AS
'Employee Name', p.ProductName,
p.Price, li.Quantity,
ROUND(p.Price * li.Quantity, 2) AS
TotalPrice,
CASE
    WHEN c.LoyaltyPoints >= 3000 THEN
30
    WHEN c.LoyaltyPoints >= 2500 THEN
25
    WHEN c.LoyaltyPoints >= 2000 THEN
20
    WHEN c.LoyaltyPoints >= 1000 THEN 10
    WHEN c.LoyaltyPoints >= 1500 THEN 15
    ELSE 0
  END AS DiscountPercent,
ROUND((p.Price * li.Quantity) *
        (1 -
          CASE
            WHEN c.LoyaltyPoints >= 1000
THEN 20
            WHEN c.LoyaltyPoints >= 500 THEN
10
            WHEN c.LoyaltyPoints >= 100 THEN
5
            ELSE 0
          END / 100.0), 2) AS DiscountedPrice
FROM Orders o
JOIN Customers c ON
o.CustomerID =
c.CustomerID
JOIN Employees e ON
o.EmployeeID =
e.EmployeeID
JOIN LineItems li ON
o.OrderID = li.OrderID
JOIN Products p ON
li.ProductID = p.ProductID
ORDER BY o.OrderDate
DESC, o.OrderID;
```

Customer Invoice

SQL Command:

```
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Employees e ON o.EmployeeID = e.EmployeeID
JOIN LineItems li ON o.OrderID = li.OrderID
JOIN Products p ON li.ProductID = p.ProductID
ORDER BY o.OrderDate DESC, o.OrderID;
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT o.OrderID, o.OrderDate, o.Payment, o.OrderStatus, o.ShippingAddress, o.PickupDate, o.DeliveryDate, CONCAT(c.FName, ' ', c.LName) AS 'Customer Name', CONCAT(e.FName, ' ', e.LName) AS 'Employee Name', p.ProductName, p.Price, li.Quantity, ROUND(p.Price * li.Quantity, 2) AS TotalPrice, CASE WHEN c.LoyaltyPoints >= 3000 THEN 30 WHEN c.LoyaltyPoints >= 2500 THEN 25 WHEN c.LoyaltyPoints >= 2000 THEN 20 WHEN c.LoyaltyPoints >= 1000 THEN 10 WHEN c.LoyaltyPoints >= 1500 THEN 15 ELSE 0 END AS DiscountPercent, ROUND((p.Price * li.Quantity) * (1 - CASE WHEN c.LoyaltyPoints >= 1000 THEN 20 WHEN c.LoyaltyPoints >= 500 THEN 10 WHEN c.LoyaltyPoints >= 100 THEN 5 ELSE 0 END / 100.0), 2) AS DiscountedPrice FROM Orders o JOIN Customers c ON o.CustomerID = c.CustomerID JOIN Employees e ON o.EmployeeID = e.EmployeeID JOIN LineItems li ON o.OrderID = li.OrderID JOIN Products p ON li.ProductID = p.ProductID ORDER BY o.OrderDate DESC, o.OrderID;

Output:

OrderID	OrderDate	Payment	OrderStatus	ShippingAddress	PickupDate	DeliveryDate	Customer Name	Employee Name	ProductName	Price	Quantity	TotalPrice	DiscountPercent	DiscountedPrice
192	2025-05-08 23:18:58	Credit Card	Ready for Pickup	78 Smart Way	2025-05-08 23:18:58	2025-05-11 23:18:58	Ella Young	Bob Williams	Holo-Projector Coffee Table	4500.00	1	4500.00	20	3600.00
192	2025-05-08 23:18:58	Credit Card	Ready for Pickup	78 Smart Way	2025-05-08 23:18:58	2025-05-11 23:18:58	Ella Young	Bob Williams	Adjustable Heated Massage Chair	499.99	1	499.99	20	399.99
192	2025-05-08 23:18:58	Credit Card	Ready for Pickup	78 Smart Way	2025-05-08 23:18:58	2025-05-11 23:18:58	Ella Young	Bob Williams	AI-Powered Bed	8000.00	1	8000.00	20	6400.00
191	2025-05-08 23:18:26	Debit Card	Shipped	56 Tech Blvd	null	2025-05-13 23:18:26	Elijah Lee	Eva Davis	Self-Cleaning Table	645.00	1	645.00	10	516.00
191	2025-05-08 23:18:26	Debit Card	Shipped	56 Tech Blvd	null	2025-05-13 23:18:26	Elijah Lee	Eva Davis	Dresser with Touchscreen Mirror	1500.00	2	3000.00	10	2400.00
190	2025-05-08 23:16:54	Credit Card	Shipped	123 TechNest Ave	null	2025-05-13 23:16:54	Tom Holland	David Miller	Chair with Built-in Speakers	400.00	1	400.00	20	320.00
190	2025-05-08 23:16:54	Credit Card	Shipped	123 TechNest Ave	null	2025-05-13 23:16:54	Tom Holland	David Miller	Wireless Charging Nightstand	150.00	3	450.00	20	360.00
190	2025-05-08 23:16:54	Credit Card	Shipped	123 TechNest Ave	null	2025-05-13 23:16:54	Tom Holland	David Miller	Smart Office Chair with Speakers	220.00	2	440.00	20	352.00
174	2025-04-30 15:00:00	Credit Card	Ready for Pickup	null	2025-04-30 18:30:00	null	Luna Moore	Bob Williams	Holo-Projector Coffee Table	4500.00	2	9000.00	10	7200.00
164	2025-04-28 09:30:00	Cash	Picked Up	null	2025-04-28 13:00:00	null	Jennifer Lawrence	Luna Martell	Wireless Charging Desk Mat	45.00	2	90.00	20	72.00
173	2025-04-12 12:15:00	Debit Card	Picked Up	null	2025-04-12 16:00:00	null	James Hernandez	Eva Davis	Dresser with Touchscreen Mirror	1500.00	1	1500.00	0	1350.00
163	2025-04-10 13:45:00	Credit Card	Ready for Pickup	null	2025-04-10 17:00:00	null	Chris Evans	Ben Parker	Ergonomic Foot Rest with Heater	55.00	1	55.00	10	44.00
162	2025-04-07 11:30:00	Debit Card	Picked Up	null	2025-04-07 16:00:00	null	Scarlett Johansson	Isabella Reed	Smart Ambient Lamp	69.99	2	139.98	20	111.98
161	2025-04-01 14:00:00	Cash	Ready for Pickup	null	2025-04-01 18:00:00	null	Ryan Reynolds	Isabella Reed	Portable Neck Massage Pillow	59.99	1	59.99	10	47.99
168	2025-03-18 10:00:00	Debit Card	Picked Up	null	2025-03-18 13:30:00	null	Emma Clark	Isabella Reed	Adjustable Laptop Stand with Cooling Fan	42.50	3	127.50	10	102.00
175	2024-09-22 08:45:00	Cash	Shipped	17 Fir St	null	2024-09-28 09:00:00	Henry Baker	David Miller	Transforming Dining Table	4000.00	1	4000.00	20	3200.00
158	2024-07-11 14:15:00	Credit Card	Delivered	101 Main St	null	2024-07-15 16:00:00	Zendaya Coleman	David Miller	Cooling Gel Pillow	39.99	1	39.99	0	35.99
169	2024-06-30 13:00:00	Cash	Processing	78 Aspen Blvd	null	null	Noah Davis	David Miller	Multi-Port Smart Charging Station	65.00	1	65.00	10	52.00
179	2024-05-10 14:20:00	Cash	Delivered	88 Sequoia Blvd	null	2024-05-15 13:00:00	Harper King	David Miller	Adjustable Heated Massage Chair	499.99	1	499.99	0	449.99
157	2024-05-03 09:10:00	Debit Card	Processing	65 Birch Blvd	null	null	Michael Brown	David Miller	Aromatherapy Pillow	34.99	3	104.97	0	94.47
183	2024-04-04 09:00:00	Debit Card	Processing	50 Elm St	null	null	Tom Holland	Eva Davis	Portable Neck Massage Pillow	59.99	1	59.99	20	47.99
165	2024-03-15 16:30:00	Debit Card	Shipped	500 River Rd	null	2024-03-20 11:00:00	Dwayne Johnson	David Miller	Bedframe with Built-in Projector	1100.00	1	1100.00	10	880.00
189	2024-03-10 09:45:00	Cash	Processing	88 Oak Dr	null	null	Grace Hawkins	Bob Williams	Dresser with Touchscreen Mirror	1500.00	1	1500.00	0	1350.00
178	2024-02-20 12:00:00	Credit Card	Shipped	77 Magnolia St	null	2024-02-25 14:00:00	Sebastian Wright	Eva Davis	Mood-Enhancing Chair	1200.00	3	3600.00	20	2880.00
154	2024-02-14 11:45:00	Debit Card	Ready for Pickup	null	2024-02-14 16:00:00	null	Evelyn Reyes	Eva Davis	Wireless Charging Nightstand	150.00	1	150.00	20	120.00
156	2024-01-20 12:00:00	Cash	Delivered	91 Willow Way	null	2024-01-26 14:00:00	Olivia Rodriguez	Bob Williams	LED Sleep Mask	49.99	1	49.99	10	39.99

Calculation 1

This query calculates Minor Employment Rule

- Find under-18 employees.
- SELECT EmployeeID, FName, LName, DOB,
- TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age,
- CASE
- WHEN TIMESTAMPDIFF(YEAR, DOB, CURDATE()) < 18 THEN 'Minor - Part-Time Only'
- ELSE 'OK'
- END AS EmploymentStatus
- FROM Employees;

Calculation 1 (cont'd)

SQL Statement: SELECT EmployeeID, FName, LName, DOB, TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age, CASE WHEN TIMESTAMPDIFF(YEAR, DOB, CURDATE()) < 18 THEN 'Minor - Part-Time Only' ELSE 'OK' END AS EmploymentStatus FROM Employees;

Output:

EmployeeID	FName	LName	DOB	Age	EmploymentStatus
1	John	Doe	1985-06-15	39	OK
2	Jane	Smith	1990-09-22	34	OK
3	Alice	Johnson	1992-03-05	33	OK
4	Bob	Williams	1987-11-30	37	OK
5	Charlie	Brown	1983-02-13	42	OK
6	David	Miller	1995-12-10	29	OK
7	Eva	Davis	1989-07-22	35	OK
8	Frank	Garcia	1980-01-19	45	OK
9	Grace	Martinez	1998-05-29	26	OK
10	Hannah	Lopez	1993-04-16	32	OK
11	Isabella	Reed	1991-10-08	33	OK
12	Liam	Nguyen	1996-06-18	28	OK
13	Sophia	Allen	1986-12-02	38	OK
14	Noah	Lee	1994-03-25	31	OK
15	Chloe	White	1999-01-14	26	OK
16	Ethan	Baker	1984-07-11	40	OK
17	Avery	Hernandez	1993-08-07	31	OK
18	Jackson	Wright	1997-05-30	27	OK
19	Lily	Young	1988-09-13	36	OK
20	Mason	Scott	1990-02-05	35	OK
21	Ben	Parker	2008-07-15	16	Minor - Part-Time Only
22	Luna	Martell	2009-11-22	15	Minor - Part-Time Only
23	Marcus	Thompson	1970-04-22	55	OK
24	Diane	Brooks	1968-11-02	56	OK
25	Carlos	Rivera	1975-09-09	49	OK
26	Sam	Taylor	1970-08-15	54	OK

Calculation 2

Employee PTO

- Calculates the total PTO that each employee has
- The company will use to make sure no employee is violating Rule 11- Total Time Off Limitations
- SELECT
- EmployeeID,
- FName,
- LName,
- (SickTime + VacationTime + PersonalTime) AS TotalPTO
- FROM Employees;

SQL Statement: SELECT EmployeeID, FName, LName, (SickTime + VacationTime + PersonalTime) AS TotalPTO FROM Employees;

Output:

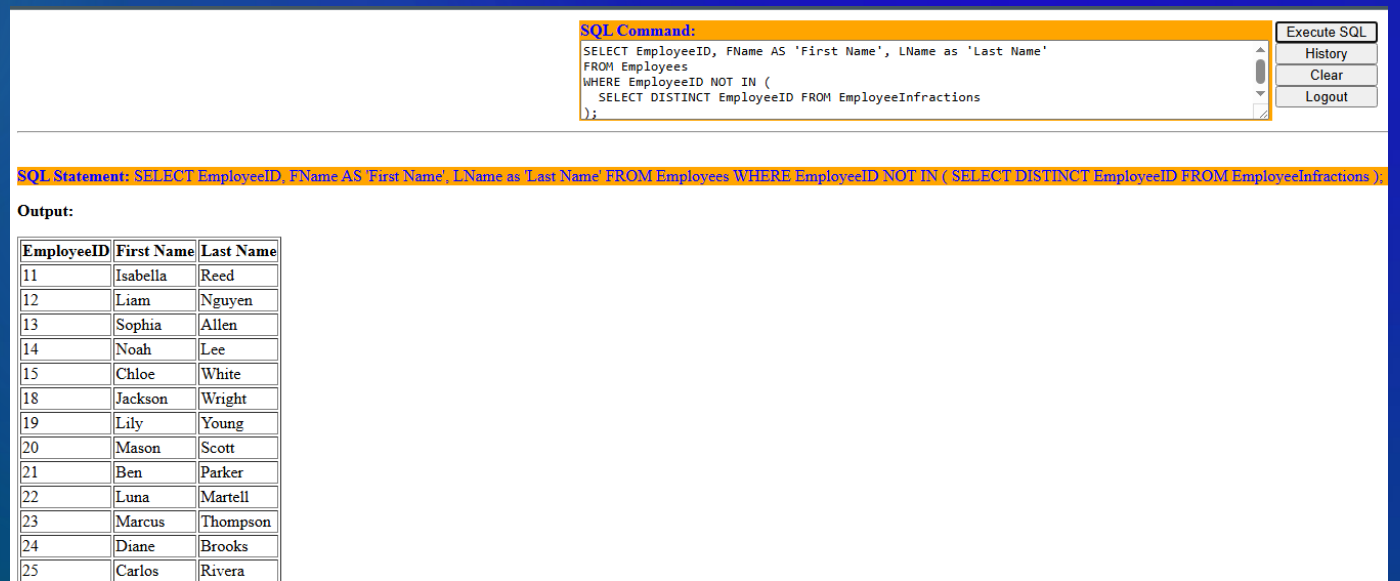
EmployeeID	FName	LName	TotalPTO
1	John	Doe	25
2	Jane	Smith	21
3	Alice	Johnson	25
4	Bob	Williams	16
5	Charlie	Brown	17
6	David	Miller	16
7	Eva	Davis	13
8	Frank	Garcia	16
9	Grace	Martinez	23
10	Hannah	Lopez	24
11	Isabella	Reed	13
12	Liam	Nguyen	26
13	Sophia	Allen	25
14	Noah	Lee	23
15	Chloe	White	15
16	Ethan	Baker	21
17	Avery	Hernandez	13
18	Jackson	Wright	25
19	Lily	Young	20
20	Mason	Scott	24
21	Ben	Parker	0
22	Luna	Martell	0
23	Marcus	Thompson	21
24	Diane	Brooks	29
25	Carlos	Rivera	16
26	Sam	Taylor	14

Sub Queries 1

No Points

- This query will display all employees with no points on their record.
- The company will use this information to reward the employees with good attendance and dress code as per **rules 6 and 8.**

```
SELECT EmployeeID, FName AS 'First Name', LName as 'Last Name'
FROM Employees
WHERE EmployeeID NOT IN
(SELECT DISTINCT EmployeeID FROM EmployeeInfractions);
```



SQL Command:

```
SELECT EmployeeID, FName AS 'First Name', LName as 'Last Name'
FROM Employees
WHERE EmployeeID NOT IN (
  SELECT DISTINCT EmployeeID FROM EmployeeInfractions
);
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT EmployeeID, FName AS 'First Name', LName as 'Last Name' FROM Employees WHERE EmployeeID NOT IN (SELECT DISTINCT EmployeeID FROM EmployeeInfractions);

Output:

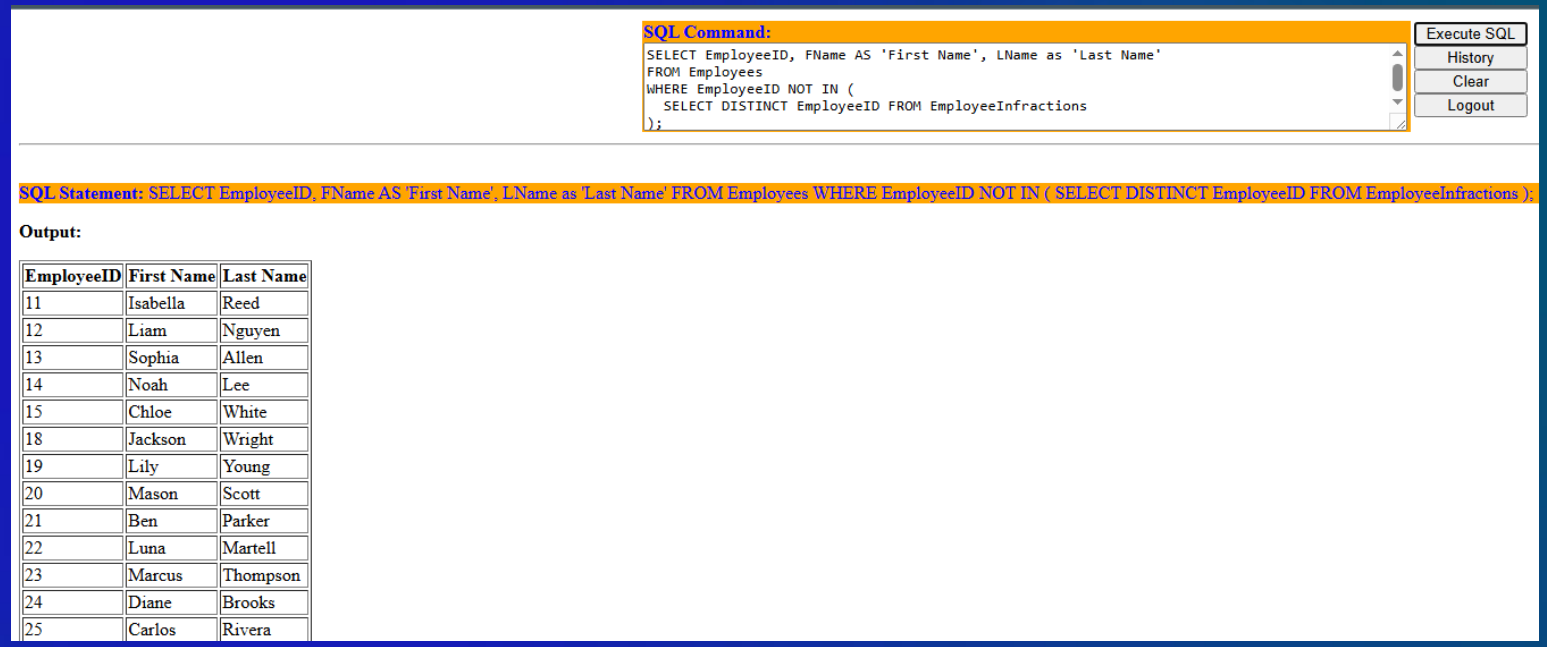
EmployeeID	First Name	Last Name
11	Isabella	Reed
12	Liam	Nguyen
13	Sophia	Allen
14	Noah	Lee
15	Chloe	White
18	Jackson	Wright
19	Lily	Young
20	Mason	Scott
21	Ben	Parker
22	Luna	Martell
23	Marcus	Thompson
24	Diane	Brooks
25	Carlos	Rivera

Sub Queries 2

Above Average Orders

- This query will display the customer names who ordered more than our usual customer.
- The company will use this information to track our reoccurring customers.

```
SELECT CustomerID, FName 'First Name', LName 'Last Name'
FROM Customers
WHERE CustomerID IN (
  SELECT CustomerID
  FROM Orders
  GROUP BY CustomerID
  HAVING COUNT(*) > (
    SELECT AVG(OrderCount)
    FROM (
      SELECT COUNT(*) AS OrderCount
      FROM Orders
      GROUP BY CustomerID
    ) AS OrderCounts
  )
);
```



SQL Command:

```
SELECT EmployeeID, FName AS 'First Name', LName as 'Last Name'
FROM Employees
WHERE EmployeeID NOT IN (
  SELECT DISTINCT EmployeeID FROM EmployeeInfractions
);
```

Execute SQL | History | Clear | Logout

SQL Statement: SELECT EmployeeID, FName AS 'First Name', LName as 'Last Name' FROM Employees WHERE EmployeeID NOT IN (SELECT DISTINCT EmployeeID FROM EmployeeInfractions);

Output:

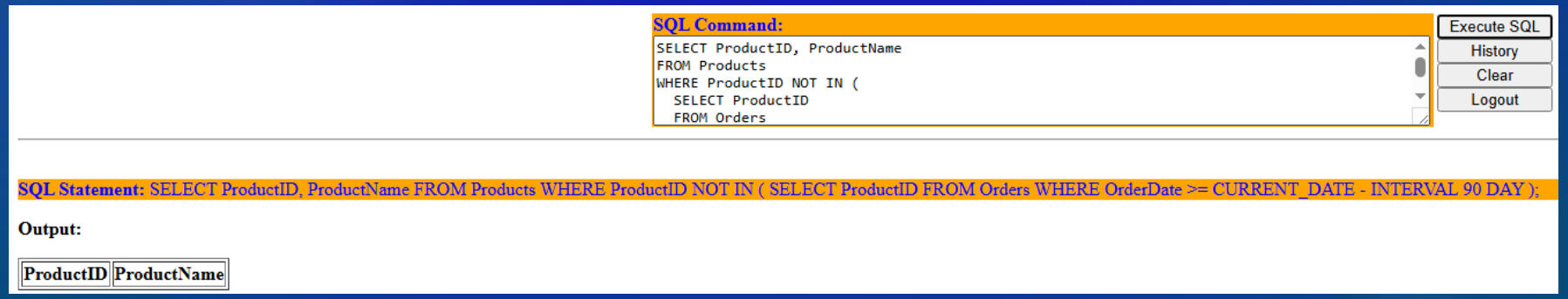
EmployeeID	First Name	Last Name
11	Isabella	Reed
12	Liam	Nguyen
13	Sophia	Allen
14	Noah	Lee
15	Chloe	White
18	Jackson	Wright
19	Lily	Young
20	Mason	Scott
21	Ben	Parker
22	Luna	Martell
23	Marcus	Thompson
24	Diane	Brooks
25	Carlos	Rivera

Sub Queries 3

Product Not Selling

- This query will display all products that haven't sold in the last 3 months.
- The company will use this information to potentially lower prices or alter product to increase sales.

```
SELECT ProductID, ProductName
FROM Products
WHERE ProductID NOT IN (
  SELECT ProductID
  FROM Orders
  WHERE OrderDate >= CURRENT_DATE – INTERVAL 90 DAY
);
```

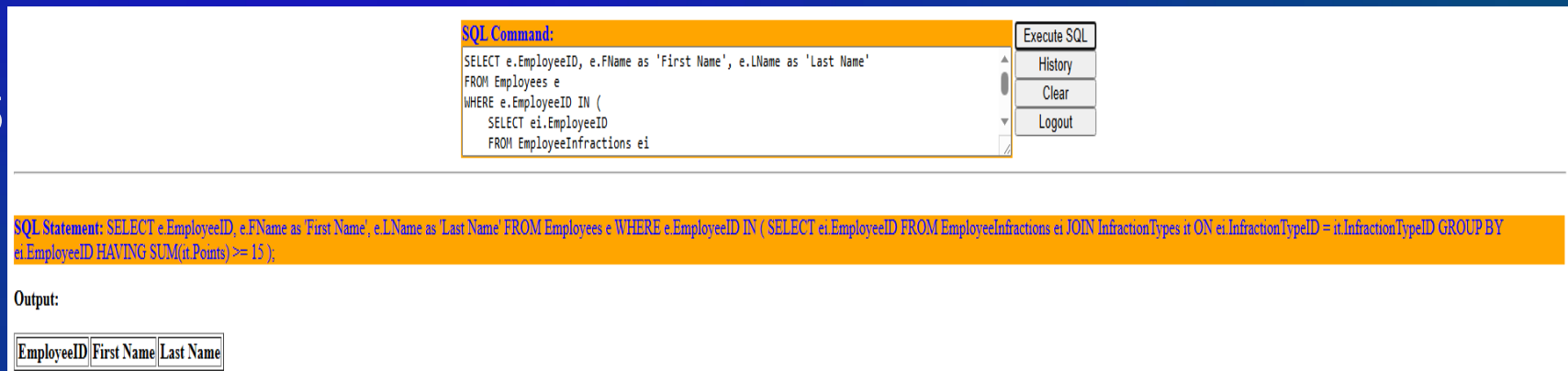


Sub Queries 4

Fired Based off Points

- This query will display any employees with over 15 points.
- The company will use this information to fire the employee as per **rule 8** that states 15 points is a termination.

```
SELECT e.EmployeeID, e.FName as 'First Name', e.LName as 'Last Name'
FROM Employees e
WHERE e.EmployeeID IN (
    SELECT ei.EmployeeID
    FROM EmployeeInfractions ei
    JOIN InfractionTypes it ON ei.InfractionTypeID = it.InfractionTypeID
    GROUP BY ei.EmployeeID
    HAVING SUM(it.Points) >= 15
);
```



Sub Queries 5

Popular Product

- This query will display any product with more than 5 sales.
- The company will use this information to advertise these products as popular hit items.

```
SELECT p.ProductID, p.ProductName,
      (SELECT SUM(li.Quantity)
       FROM LineItems li
       WHERE li.ProductID = p.ProductID) AS TotalOrdered
FROM Products p
WHERE (SELECT SUM(li.Quantity)
FROM LineItems li
WHERE li.ProductID =
p.ProductID) > 5;
```



SQL Statement: SELECT p.ProductID, p.ProductName, (SELECT SUM(li.Quantity) FROM LineItems li WHERE li.ProductID = p.ProductID) AS TotalOrdered FROM Products p WHERE (SELECT SUM(li.Quantity) FROM LineItems li WHERE li.ProductID = p.ProductID) > 5;

Output:

ProductID	ProductName	TotalOrdered
4	Chair with Built-in Speakers	8